

Complejidad Temporal y Patrones de Diseño de Software

Práctica 1

Repaso de POO. Interfaces

Ejercicio 1

Implemente la siguiente interface (**Comparable**) que representa cualquier objeto que sabe compararse con otro objeto de su "mismo tipo":

Comparable

sosIgual(Comparable)	Devuelve verdadero si el objeto que recibe el mensaje es el mismo que el "comparable" recibido por parámetro, devuelve falso en caso contrario
sosMenor(Comparable)	Devuelve verdadero si el objeto que recibe el mensaje es más chico que el "comparable" recibido por parámetro, devuelve falso en caso contrario
sosMayor(Comparable)	Devuelve verdadero si el objeto que recibe el mensaje es más grande que el "comparable" recibido por parámetro, devuelve falso en caso contrario

Nota: asumimos que el elemento que recibe el mensaje y el objeto recibido por parámetro son del "mismo tipo".

Ejercicio 2

Implemente la clase **Visualizacion**, que representa la cantidad de reproducciones que tuvo un contenido de la plataforma:

Visualizacion

cantidad	Es una variable que almacena un número entero
constructor(c)	Recibe un valor "c" y lo almacena en la variable "cantidad"
getCantidad	Devuelve la variable "cantidad"

Haga que la clase **Visualizacion** implemente la interface **Comparable**.

*Nota: en C#, en los tres métodos a implementar, castee el parámetro recibido a **Visualizacion**.*

Ejercicio 3

Implemente la siguiente interface que representa objetos que almacenan comparables:

Coleccionable

cuantos	Devuelve la cantidad de elementos comparables que tiene el coleccionable
minimo	Devuelve el elemento de menor valor de la colección
maximo	Devuelve el elemento de mayor valor de la colección
agregar(Comparable)	Agrega el comparable recibido por parámetro a la colección que recibe el parámetro
contiene(Comparable)	Devuelve verdadero si el comparable recibido está incluido en la colección y falso en caso contrario

Ejercicio 4

Implemente las clases **Pila** y **Cola** (vistas en materias anteriores) y haga que las dos clases implementen la interface **Coleccionable**.

Ejercicio 5

Implemente una función **llenar** que reciba un **Coleccionable** y que le agregue 20 comparables elegidos al azar.

```
llenar(Coleccionable)
    repetir 20 veces
        comparable = new Visualizacion(cantidad_elegida_al_azar)
        coleccionable.agregar(comparable)
```

Ejercicio 6

Implemente una función **informar** que reciba un **Coleccionable** e imprima por consola la cantidad de elementos, el mínimo, el máximo y si contiene, o no, un valor leído por teclado.

```
informar(Coleccionable)
    imprimir (coleccionable.cuantos())
    imprimir (coleccionable.minimo())
    imprimir (coleccionable.maximo())
    comparable = new Visualizacion(leer_por_teclado)
    si (coleccionable.contiene(comparable))
        imprimir("El elemento leído está en la colección")
    sino
        imprimir("El elemento leído no está en la colección")
```

Ejercicio 7

Implemente un módulo **main** que cree una **Pila** y una **Cola**, las llene, e invoque **informar** con ambas.

```
main
    pila = new Pila()
    cola = new Cola()
    llenar(pila)
    llenar(cola)
    informar(pila)
    informar(cola)
```

Ejercicio 8

Cree la clase **Catalogo**:

Catalogo

pila	Es una variable que almacena una pila
cola	Es una variable que almacena una cola
constructor(p, c)	Recibe una pila "p" y una cola "c" y las almacena

Haga que **Catalogo** implemente **Coleccionable**, donde:

- **cuantos**: cantidad de elementos de ambos coleccionables
- **minimo / maximo**: entre ambos coleccionables
- **agregar**: no hace nada
- **contiene**: verdadero si el comparable está en alguna de las dos colecciones

Ejercicio 9

Modifique el **main** para crear un **Catalogo** e informar con esta colección.

```
main
    pila = new Pila()
    cola = new Cola()
    catalogo = new Catalogo(pila, cola)
    llenar(pila)
    llenar(cola)
    informar(pila)
    informar(cola)
    informar(catalogo)
```

Ejercicio 10

Implemente la clase abstracta **Perfil**:

Perfil

nombre	string
id	número
constructor(n, i)	Recibe nombre e id y los almacena
getNombre getId	Devuelven las variables correspondientes

Haga que **Perfil** implemente **Comparable**. Compare por id o por nombre, a elección.

Ejercicio 11

Implemente la clase **Suscriptor**, subclase de **Perfil**:

Suscriptor : Perfil

mesesDeSuscripcion	número
horasVistas	número
constructor(n, i, c, h)	Recibe nombre, id, meses de suscripción y horas vistas
getMesesDeSuscripcion getHorasVistas	Devuelven las variables correspondientes

Ejercicio 12

Implemente una función **llenarSuscriptores** que reciba un **Coleccionable** y le agregue 20 suscriptores al azar.

```
llenarSuscriptores(Coleccionable)
    repetir 20 veces
        comparable = new Suscriptor(nombre_al_azar,
                                     id_al_azar,
                                     mesesDeSuscripcion_al_azar,
                                     horasVistas_al_azar)
        coleccionable.agregar(comparable)
```

Ejercicio 13

Modifique el **main**:

```
main
    pila = new Pila()
    cola = new Cola()
    catalogo = new Catalogo(pila, cola)
    llenarSuscriptores(pila)
    llenarSuscriptores(cola)
    informar(catalogo)
```

Ejercicio 14

Reimplemente los métodos de **Comparable** en **Suscriptor** para que compare por meses de suscripción o por horas vistas, a elección. Ejecute nuevamente el **main** del ejercicio anterior para comprobar el correcto funcionamiento.

Este ejercicio, y todos los anteriores que dependen de él, debe ser entregado en el aula virtual del campus (Entrega 1).